CprE 4870/5870
Lab 6: Multithreading, Tiling, and SIMD

*"Some people, when confronted with a problem, think, "I know, I'll use threads," and then two they hav erpoblesms." –**Ned Batchelder***

1 Learning Objectives 🎯

By the end of this lab you should be able to

- Quantify the performance of a deep neural network classification model running on general-purpose hardware (CPUs).
- Evaluate software-based optimizations of a deep neural network classification models running on a CPU leveraging hardware knowledge (data re-use from locality, SIMD execution, multi-threaded execution, and quantization).

2 Pre-lab ✅

2.1 Lab Checklist

- Complete lab 2 (Or at least have a basic understanding of implementing neural network layers).
- Have completed the C++ environment setup demonstrated in lab 2.

2.2 Intro

In this lab, we are going to be re-implementing and optimizing our previously implemented ML model in C++. We will be exploring cache coherent operations an data access/reuse, SIMD operations, and Multi-threading. Note: While we could also include quantization into this mix, including quantization support for these implementations is OPTIONAL.

This lab should realistically take your group **one week** to complete.

3 Computational Acceleration 📊

3.1 Tiling

As shown in figure 1 significant performance gains can be leveraged by optimizing memory accesses.

1. Identify Cache sizes: Identify the cache sizes of L1 data, L1 instruction, L2 and, L3 cache of the Zedboard.

Memory	Size	Latency	Bandwidth
L1 cache	32 KB	1 nanosecond	1 TB/second
L2 cache	256 KB	4 nanoseconds	1 TB/second Sometimes shared by two cores
L3 cache	8 MB or more	10x slower than L2	>400 GB/second
MCDRAM		2x slower than L3	400 GB/second
Main memory on DDR DIMMs	4 GB-1 TB	Similar to MCDRAM	100 GB/second

Figure 1: Memory Performance from Intel

2. Identify & reorder memory accesses: Going back to the layer that contributed most to the computation latency identify the memory access patterns in the core computation. Do they leverage caches? If not reorder the matrices used in the core computation to increase the probability of hitting the cache i.e. more temporal and spatial reuse. Report the speed up obtained. Please profile at least 100 inferences of the program for both before and after reordering memory accesses version. Plot the weight/filer, input, and output feature map access patterns of your cache optimized implementation and include these 3 charts in your report. Explain how these graphs demonstrate that your data re-use pattern is optimal for cache coherency. Note: it will likely be easiest to construct these graphs in Python using matplot lib.

   It is strongly encouraged that you consider reordering the static data files (weights, activations) for this part and exporting new files from the lab 01 Jupyter Notebook. This will be a useful skill to have for lab 06.

3. Tiling in Cache is used to improve performance by enabling cache reuse. figure 2 is an example of cache tiling. It shows two big arrays A and B being blocked into smaller rectangular chunks so that the total size of two A and B chunks can fit inside the cache. For further details on tiling you may refer the link or section 4.2 from the book.

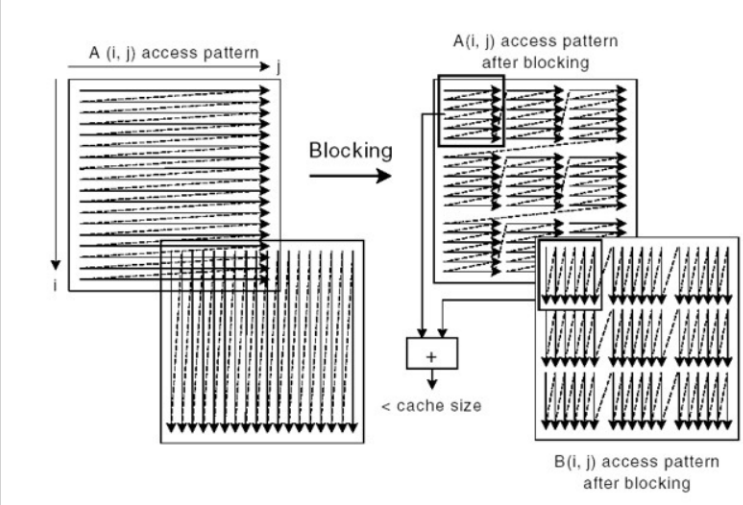


Figure 2: Cache Tiling Intel

Re-write the layer contributing most to the computation latency using tiling. Report the speed up you obtain over a non-tiled versions the non optimized version on the zedboard. Please run at least 100 iterations of the program for both tiled and non-tiled version.

Experiment with the block size of 2, 4, 8, 16, 32. Report the speedup obtained over the baseline for each of the block size. Plot the single image inference latency for each of the block size. Which block size did you find optimal? State the possible reasons.

3.2 Multi-threading (Optional)

The computation latency could be further shortened by leveraging multi-core processing using multi-threading. Pthreads or POSIX threads is a standard C programming interface for UNIX systems to enable multithreading. You may refer the documentation for `std::thread` and `std::jthread` here or this basic tutorial tutorial to get a quick intro to start using the pthread library in C++. You will need to include the pthread header in your code to use these libraries (`include <thread>`).

As the zedboard is bare-metal, there is no multithreading support. **Therefore this section will be performed on the lab computer only.**

Take the layer that you have identified contributes most to the inference latency and using the pthreads library, write a multi-threaded implementation to parallelize the computation of that layer. And report the following:

1. Vary the number of threads from 2, 4, 8, 16, 32, 64. Report the speed up obtained in addition to enabling also enabling SIMD instructions (`-march-native`).

2. Plot the inference latency vs the number of threads. What do you think are the optimal number of threads and why? Utilize your knowledge of the computation, the hardware architecture of the lab computers, and the operations the threads are performing.

3.3 SIMD (Optional)

SIMD allows to process multiple data in single instruction compared to traditional SISD figure 3

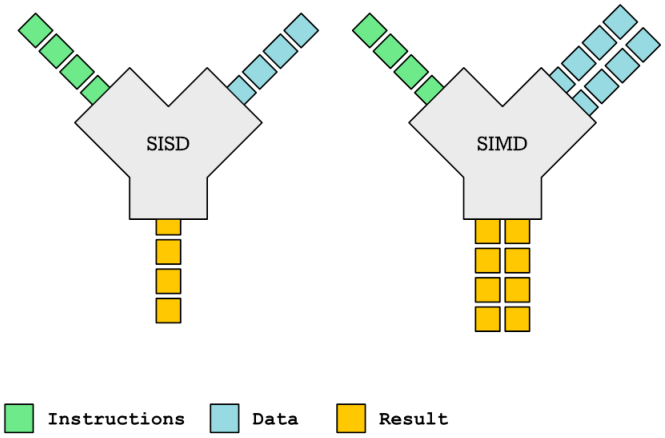


Figure 3: SISD vs SIMD

Intel provides various SIMD extensions as shown in figure 4

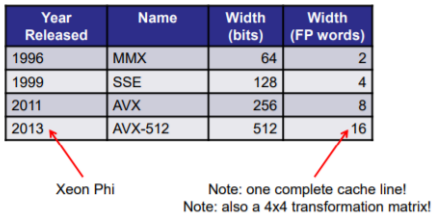


Figure 4: Intel SIMD

Using your test inputs (at least one, but a small batch of inputs is also valid), report the latency per inference for the following optimizations:

1. No SIMD, but still with `-O3` optimizations (`DEBUG=false`). NOTE: All tests should be run with `-O3` optimizations enabled.
2. SIMD (basic): Enable the SSE2 SIMD instructions `-msse2`.
3. SIMD (modern): Enable the AVX SIMD instructions `-mavx`.
4. SIMD: AVX512 (OPTIONAL) Similarly enable `AVX512` and analyze the assembly code? Report the changes in the instructions you observe compared to Non-optimized and SSE2 version. Try to quantify the obtained speed up with the changes observed in the assembly code. You should be able to find instructions like vaddss, vmulss etc in the assembly. See the `-mavx` instructions documented here for how to enable them.
5. SIMD (all): Enable the all SIMD instructions supported `-march=native`. Note: this compilation must be on on the same computer you are running your tests on, as it will determine the capabilities of the current system that can change from generation to generation.

Additionally, try configuring the zedboard to enable use of its SIMD instructions. This will require changing build flags which may be easiest to edit by opening your workspace folder in GUI Vitis. How does enabling SIMD support affect the latency when running on the zedboard?

Analyze the assembly code? Report the changes in the instructions you observe compared to Non-optimized version. Try to quantify the obtained speed up with the changes observed in the assembly code. The assembly code can be generated using `ASM=true` (tools such as Compiler Explorer can also be helpful, although be aware that you may be running on a completely different server architecture, so your report should be based on your lab machine results). You should be able to find instructions like addss, mulss etc in the assembly code for some of your computational operations. Include a snippet of assembly code in your report where SIMD instructions are generated for each configuration tested.

3.4 Need for Speed

Take all that you have learned from the optimizations implemented today and construct a single implementation that combines the best of them. Report how does the best latency you received compare to the fastest recorded from Tensorboard? Do we get anywhere close to the same performance? Report your best latency and best throughput (on at least 100 images) to be entered into the class competition to receive 5 extra credit (5 for latency and 5 for throughput) lab points (All runs must be done on the lab computers).

4 Above & Beyond 🚀

- Explore SIMT (GPU) architectures and how we can better parallelize these operations using specialized hardware.
- Specifically utilize SIMD/Vector instructions through compiler (gcc) built-in functions.
- Use a compiler extension library (ex. OpenMP, OpenACC, Intel TBB) to parallelize your loops.
- Support Quantization for some or all of the optimization techniques explored.
- Enter the class inference latency and throughput contest.

5 Submission Instructions 🗂️

Below is a list of the required turn-in components for the lab to be submitted to Canvas. Only the final submission will be graded, although you are free to resubmit as many times as you please. All submissions must be complete. Even slight changes require an complete re-submission, **NOT** a partial, single file, change. Naming conventions for each file are also provided below.

5.1 Turn-In Checklist

- PDF of your lab document, either from the template or your own format containing all the required lab components (`lab6_report_<Lab Partners Number>.pdf`).

5.2 File Structure

```
Canvas
├── lab6_report_<Lab Partners Number>.pdf
├── lab6_code_<Lab Partners Number>.zip/.tgz
    └── ...
```